So there you have it. Our Redis server is up and running, and we can connect to it from a separate container. Note that we're using `docker-compose run`—rather than `exec`—specifically so that the `redis-cli` runs in a new, separate container, albeit based on the same `redis` image. This shows that we're able to access the Redis server from a *different* container.

But, hang on a sec! Aren't containers supposed to be isolated? How come we were able to connect from the container running `redis-cli` to the container running the `redis` server?

Good question. Let's explore this in the next section.

How Containers Can Talk to Each Other

If two containers are isolated, independent processes, how come, as we just saw, that they are able to talk to one another? While it's true that the *code and processes* running in a container are sandboxed, that does not mean the container has no way to communicate with the outside world. If containers could not communicate, we would not be able to connect them together to create a powerful, connected system of services that together make up our application.

If you remember back to Launching Our App, on page 47, we said that `docker-compose up` creates a new network for the app. By default, all containers for our app are connected to the app's network and can communicate with each other. This means that our containers, just like a physical or virtual server, can communicate outside themselves using TCP/IP networking.

Let's list our currently defined networks using the command:

```
$ docker network ls
```

You should see some output similar to the following:

```
NETWORK ID          NAME                DRIVER        SCOPE
128925dfad81        bridge              bridge        local
5bd7167263e8        host                host          local
e2af02026928        myapp_default       bridge        local
d1145155d62a        none                null          local
```

The first network called `bridge` is a legacy network to provide backwards compatibility with some older Docker features—we won't be using it now that we've switched to Compose. Similarly, the `host` and `none` networks are special networks that Docker sets up that we don't need to care about.

The network we do care about is called `myapp_default`—this is our app's dedicated network that Compose created for us (Compose uses the `<appname>_default`